

CAP455: OBJECT ORIENTED PROGRAMMING USING C++

The Proposal For

Swimming Pool Lane Management

Master of Computer Applications (MCA)
Academic Session: 2026–2027
Submission: CA1 Proposal

Submitted By:

Prajapati Vikash Rajaram

Reg No.: 12600782

Roll No.: RD1P2635A30

Submitted To:

Mr. Adarsh Kumar Singh

GitHub Repo. Link.: https://github.com/vikash-r-prajapati/Project-CAP455-C-

TABLE OF CONTENTS

Content	Page
1. Executive Summary	3
2. Problem Statement & Proposed Solution	3
3. Class and Entity Design	4
4. Unique ID & Duplicate Prevention	4
5. Data Dictionary: Attributes	5
6. Data Dictionary: Functions	5
7. OOP Concept Mapping	6
8. UML Class Diagram	7
9. System Flowchart	8
10. Complete Source Code	9–11
11. Sample Output	12

1. EXECUTIVE SUMMARY

Swimming Pool System is a menu-driven console application developed in C++. It maintains swimmer registration records in a fixed-size array and provides options to register a swimmer, view all registered swimmers, search for a swimmer by ID, display the total number of swimmers and exit.

The Swimmer class represents an individual swimmer. Its private data members store swimmer ID, name, age and phone number. A default constructor initializes the members. The setDetails() member function assigns the entered values and increments the static totalSwimmers counter. The inline displayDetails() function displays a swimmer record. The static getTotalSwimmers() function returns the shared count, and the friend function showSwimmer() displays the swimmer's name and ID using private data.

The program creates an array of 200 Swimmer objects. It checks for a positive, non-duplicate ID and a non-empty name before registration. Age is checked to be a positive integer. Records are held in memory only and are not saved after the program exits.

2. PROBLEM STATEMENT & PROPOSED SOLUTION

2.1 Background and Problem Statement

A swimming pool or training facility may need to maintain basic information about its registered swimmers. Paper-based records can make it difficult to search for a particular swimmer, maintain a count of registrations or avoid duplicate identification numbers. A small computerized record system can make these basic tasks more organized.

2.2 Proposed Solution

The proposed solution is a C++ console application that stores swimmer information in Swimmer objects. A fixed array can hold up to 200 records. The menu supports registration, viewing all swimmers, searching by ID, viewing the total swimmer count and exiting. The program checks the ID for positivity and duplication and checks that the name is not empty. Age is checked to be a positive integer.

2.3 Project Objectives

- Store swimmer ID, name, age and phone number in one object.
- Register and search swimmer records through a menu.
- Prevent duplicate swimmer IDs within the current execution.
- Display all registered swimmers and the total count.
- Demonstrate constructors, encapsulation, static members, inline functions, friend functions, arrays and object pointers.

3. CLASS AND ENTITY DESIGN

3.1 Swimmer Class

Swimmer is the main entity class. Each object represents one registered swimmer. Its private attributes are swimmerId, name, age and phone. Public functions initialize, set and display details, return the ID and provide the total count.

Attribute	Type	Access	Purpose
swimmerId	int	private	Unique swimmer identification number.
name	string	private	Swimmer's name.
age	int	private	Swimmer's age in years.
phone	string	private	Contact phone number.
totalSwimmers	static int	private	Shared count of registered swimmers.
MAX_SWIMMERS	const int	global	Maximum array capacity: 200.

3.2 Main Function and Array

main() creates Swimmer swimmers[MAX_SWIMMERS], initializes count to zero and repeats the menu using a do-while loop. The count variable identifies how many array positions contain registered swimmers. Registration stores the next swimmer in swimmers[count] and increments count after setDetails() is called.

4. UNIQUE ID & DUPLICATE PREVENTION

The swimmerId member is used as the record identifier. Before registration, the program rejects an ID if it is less than or equal to zero or if isDuplicate() finds the same ID among the first count objects. The helper function isDuplicate() loops through the registered portion of the array and compares each object's ID using getId().

4.1 Duplicate Check Pseudocode

```
Read swimmer ID
If ID <= 0: reject ID
For i = 0 to count - 1:
    If swimmers[i].getId() equals ID:
        Return true (duplicate)
Return false (not duplicate)
If valid:
    Store details in swimmers[count]
    Increase count
```

Duplicate checking is limited to the records stored during the current program run. There is no file or database storage in this version.

5. DATA DICTIONARY: ATTRIBUTES

Variable	Type	Scope	Description
swimmerId	int	Swimmer private	Positive ID used to identify a swimmer.
name	string	Swimmer private	Name; checked for empty input.
age	int	Swimmer private	Age; must be greater than zero.
phone	string	Swimmer private	Phone number entered by the user.
totalSwimmers	static int	Swimmer private	Shared registration counter.
MAX_SWIMMERS	const int	Global	Array capacity set to 200.
swimmers	Swimmer[200]	main local	Fixed array storing swimmer objects.
count	int	main local	Number of registered records.
choice	int	main local	Menu selection.
ptr	const Swimmer*	case 2 local	Pointer used to traverse the array.

6. DATA DICTIONARY: FUNCTIONS

Function	Kind	Purpose
Swimmer()	Default constructor	Initializes ID and age to zero and text fields to empty.
setDetails(int,string,int,string)	Member function	Assigns swimmer data and increments totalSwimmers.
displayDetails()	Inline member function	Displays all details of the current swimmer.
getTotalSwimmers()	Static member function	Returns the class-level swimmer count.
getId()	Const member function	Returns the swimmer ID.
showSwimmer(const Swimmer&)	Friend function	Displays name and ID using private members.
isDuplicate(...)	Global helper function	Checks whether an ID already exists in the array.

6.1 Static Data Member Definition

```
int Swimmer::totalSwimmers = 0;
```

The static data member is declared inside the Swimmer class and must be defined outside the class. It is shared by all Swimmer objects.

7. OOP CONCEPT MAPPING

7.1 Class and Objects

Swimmer is the class; each registered swimmer is represented by an object in the swimmers array.

7.2 Encapsulation

The data members are private. Public member functions provide controlled access to set and display details.

7.3 Default Constructor

Swimmer() initializes swimmerId and age to zero and name and phone to empty strings.

7.4 Static Data Member

totalSwimmers is shared by all objects and counts registrations. It is initialized outside the class.

7.5 Static Member Function

getTotalSwimmers() returns the shared counter and is called using Swimmer::getTotalSwimmers().

7.6 Inline Function

displayDetails() is declared inline inside the class and displays the current object's data.

7.7 Friend Function

showSwimmer(const Swimmer& s) can read private name and swimmerId because it is declared as a friend.

7.8 Array of Objects

Swimmer swimmers[MAX_SWIMMERS] creates an array capable of storing 200 swimmer objects.

7.9 Object Pointer

const Swimmer* ptr = swimmers points to the first array element. Pointer arithmetic (ptr + i) accesses each swimmer.

7.10 this Pointer

setDetails() uses this-> to refer to the current object's data members.

7.11 Reference Parameter

The friend function accepts const Swimmer& so it refers to the original object without copying it.

7.12 Loops and Selection

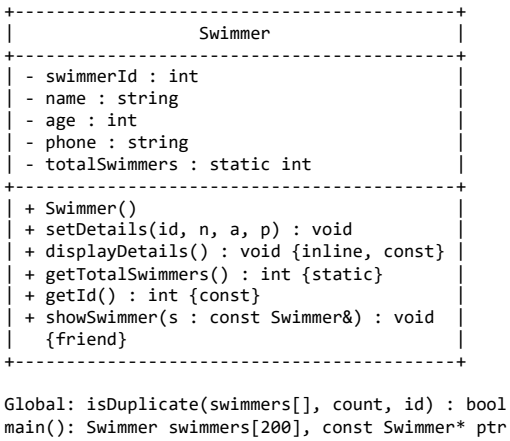
The do-while loop repeats the menu, switch selects an operation, and for loops search or display records.

7.13 Input Validation

The program checks menu input, positive IDs, duplicate IDs, non-empty names and positive ages.

8. UML CLASS DIAGRAM

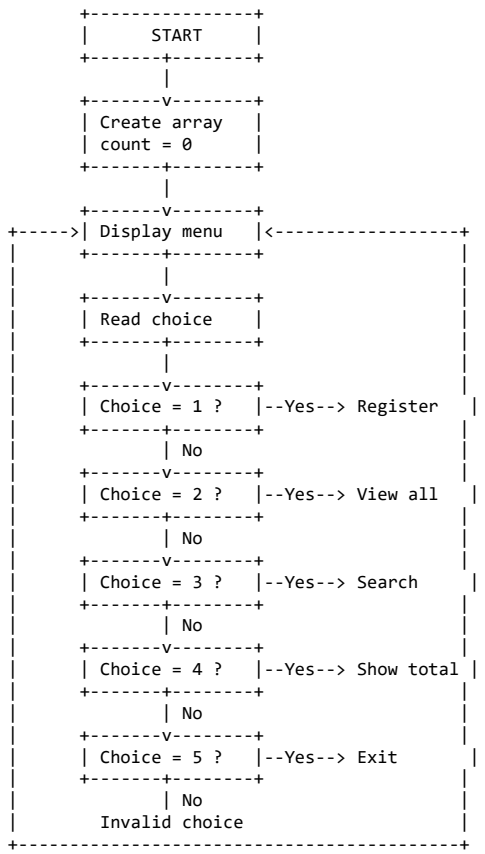
The UML represents the Swimmer class and its public and private members. The global helper function isDuplicate() operates on the array of Swimmer objects.



Global: isDuplicate(swimmers[], count, id) : bool
main(): Swimmer swimmers[200], const Swimmer* ptr

9. SYSTEM FLOWCHART

The system begins by creating the swimmer array and showing the menu. The selected operation is handled by the switch statement. After each operation, the menu is shown again until the user selects Exit.



Registration: check capacity → input ID → check positivity and duplication → input name and age → input phone → set details → increase array count.

Search: read ID → compare with each registered swimmer ID → display friend summary if found → otherwise display not-found message.

10. COMPLETE SOURCE CODE

File: main.cpp — corrected for valid C++ static-member definition.

```
#include <iostream>
#include <string>
using namespace std;

const int MAX_SWIMMERS = 200;

class Swimmer {
private:
    int swimmerId;
    string name;
    int age;
    string phone;
    static int totalSwimmers;

public:
    // Default constructor
    Swimmer() {
        swimmerId = 0;
        name = "";
        age = 0;
        phone = "";
    }

    // Set swimmer details
    void setDetails(int id, string n, int a, string p) {
        this->swimmerId = id;
        this->name = n;
        this->age = a;
        this->phone = p;
        totalSwimmers++;
    }

    // Inline function
    inline void displayDetails() const {
        cout << "\nSwimmer ID: " << swimmerId;
        cout << "\nName: " << name;
        cout << "\nAge: " << age;
        cout << "\nPhone: " << phone << endl;
    }

    // Static function
    static int getTotalSwimmers() {
        return totalSwimmers;
    }

    // Get swimmer ID
    int getId() const {
        return swimmerId;
    }

    // Friend function
    friend void showSwimmer(const Swimmer& s);
```

10. SOURCE CODE (CONTINUED)

```
};

// Initialize static data member outside the class
int Swimmer::totalSwimmers = 0;

// Friend function
void showSwimmer(const Swimmer& s) {
    cout << "\nSwimmer Name: " << s.name;
    cout << "\nSwimmer ID: " << s.swimmerId << endl;
}

// Check duplicate ID
bool isDuplicate(Swimmer swimmers[], int count, int id) {
    for (int i = 0; i < count; i++) {
        if (swimmers[i].getId() == id) {
            return true;
        }
    }
    return false;
}

int main() {
    Swimmer swimmers[MAX_SWIMMERS];
    int count = 0;
    int choice;

    do {
        cout << "\n------- SWIMMING POOL SYSTEM -----------\n";
        cout << "1. Register Swimmer\n";
        cout << "2. View All Swimmers\n";
        cout << "3. Search Swimmer\n";
        cout << "4. Total Swimmers\n";
        cout << "5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "Invalid input!\n";
            continue;
        }

        switch (choice) {
            case 1: {
                if (count == MAX_SWIMMERS) {
                    cout << "Registration is full!\n";
                    break;
                }

                int id, age;
                string name, phone;
```

10. SOURCE CODE (CONTINUED)

```
        cout << "Enter Swimmer ID: ";
        cin >> id;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "Invalid ID!\n";
            break;
        }

        if (id <= 0 || isDuplicate(swimmers, count, id)) {
            cout << "Invalid or duplicate ID!\n";
            break;
        }

        cin.ignore(1000, '\n');

        cout << "Enter Name: ";
        getline(cin, name);

        if (name.empty()) {
            cout << "Name cannot be empty!\n";
            break;
        }

        cout << "Enter Age: ";
        cin >> age;

        if (cin.fail() || age <= 0) {
            cin.clear();
            cin.ignore(1000, '\n');
            cout << "Invalid age!\n";
            break;
        }

        cout << "Enter Phone: ";
        cin >> phone;

        swimmers[count].setDetails(id, name, age, phone);
        count++;
        cout << "Swimmer registered successfully!\n";
        break;
    }

    case 2: {
        if (count == 0) {
            cout << "No swimmers found!\n";
            break;
        }

        const Swimmer* ptr = swimmers;
```

10. SOURCE CODE (CONTINUED)

```
                for (int i = 0; i < count; i++) {
                    (ptr + i)->displayDetails();
                }
                break;
            }

            case 3: {
                int id;
                bool found = false;

                cout << "Enter Swimmer ID: ";
                cin >> id;

                for (int i = 0; i < count; i++) {
                    if (swimmers[i].getId() == id) {
                        showSwimmer(swimmers[i]);
                        found = true;
                        break;
                    }
                }

                if (!found) {
                    cout << "Swimmer not found!\n";
                }
                break;
            }

            case 4:
                cout << "Total Swimmers: "
                     << Swimmer::getTotalSwimmers() << endl;
                break;

            case 5:
                cout << "Exiting Swimming Pool System...\n";
                break;

            default:
                cout << "Invalid choice!\n";
        }
    } while (choice != 5);

    return 0;
}
```

11. SAMPLE OUTPUT

Illustrative console session showing swimmer registration, listing, search, total count and exit.

```
------- SWIMMING POOL SYSTEM -----------
1. Register Swimmer
2. View All Swimmers
3. Search Swimmer
4. Total Swimmers
5. Exit
Enter choice: 1
Enter Swimmer ID: 101
Enter Name: Aman Sharma
Enter Age: 20
Enter Phone: 9876543210
Swimmer registered successfully!

------- SWIMMING POOL SYSTEM -----------
1. Register Swimmer
2. View All Swimmers
3. Search Swimmer
4. Total Swimmers
5. Exit
Enter choice: 2

Swimmer ID: 101
Name: Aman Sharma
Age: 20
Phone: 9876543210

------- SWIMMING POOL SYSTEM -----------
1. Register Swimmer
2. View All Swimmers
3. Search Swimmer
4. Total Swimmers
5. Exit
Enter choice: 4
Total Swimmers: 1

------- SWIMMING POOL SYSTEM -----------
1. Register Swimmer
2. View All Swimmers
3. Search Swimmer
4. Total Swimmers
5. Exit
Enter choice: 5
Exiting Swimming Pool System...
```

11.1 Validation Examples

- Registration is rejected when the array reaches 200 swimmers.
- A non-positive or duplicate ID is rejected.
- An empty name is rejected.
- A non-numeric or non-positive age is rejected.
- Searching for an ID that is not present displays “Swimmer not found!”

Scope and Limitations

This version stores records only in memory. It does not validate phone-number format, store data in files or databases, manage pool sessions, track attendance or process payments. These are not features of the supplied code.